Social Media Analytics

Graduation Project — Full Technical Documentation

Amazon Fine Food Reviews — Sentiment Analysis Pipeline

GitHub Repository: github.com/Mina-medhat1/ssocial-media-analytics-tasks

Dataset: Amazon Fine Food Reviews (Kaggle)

Project Overview

This document provides a comprehensive technical explanation of the Social Media Analytics Graduation Project. The project implements a complete sentiment analysis pipeline on the Amazon Fine Food Reviews dataset, progressing through five tasks: data collection and initial exploration, text preprocessing, multi-model sentiment classification, model evaluation and benchmarking, and SOTA comparison.

Task	Title	Notebook File
Task 1	Data Collection & Initial Analysis with API	task1.ipynb
Task 2	Data Cleaning & Preprocessing	task2.ipynb
Task 3	Sentiment Modeling (3 Schemes)	social_task3.ipynb
Task 4	Evaluation & Benchmarking	social_TASK4.ipynb
Task 5	SOTA Comparison (RoBERTa vs SVM)	task5_final.ipynb

Task 1: Data Collection & Initial Sentiment Analysis

This task establishes the foundation of the project by downloading the Amazon Fine Food Reviews dataset from Kaggle and performing an initial, exploratory sentiment analysis using a state-of-the-art zero-shot classification model (DeBERTa).

1.1 Environment Setup & Dataset Download

The first step is installing all required Python libraries and configuring the Kaggle API credentials to download the dataset.

```
! pip install kaggle
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d snap/amazon-fine-food-reviews
!unzip -q amazon-fine-food-reviews.zip
```

Why this step?

The Kaggle API key (kaggle.json) must be placed in the ~/.kaggle/ directory with restricted permissions (chmod 600) so the Kaggle CLI can authenticate and download the dataset securely.

1.2 Loading the Dataset

Once downloaded, the CSV file (Reviews.csv) is loaded into a Pandas DataFrame. A small preview sample of 6 rows (indices 0–5) is used for the initial DeBERTa analysis to control compute time.

```
df_hn = pd.read_csv('/content/Reviews.csv')
df_hn = df_hn.loc[0:5, :]   # Take first 6 rows for initial preview
print(df_hn.shape)
```

1.3 Feature Engineering — Combining Text Fields

To give the sentiment model more context, the Summary column (short review headline) is concatenated with the full Text column. This creates a richer Full_Text field that is used as input.

```
df_hn['Full_Text'] = df_hn['Summary'].fillna('') + ' ' +
df_hn['Text'].fillna('')
```

Why? The Summary field contains the reviewer's headline (e.g., 'Great product!'). Merging it with the full review body gives the DeBERTa model a more complete view of the reviewer's opinion.

1.4 Zero-Shot Sentiment Analysis with DeBERTa

The project uses the MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli model from Hugging Face. This is a Natural Language Inference (NLI) model that can classify text into arbitrary categories without task-specific fine-tuning — hence 'zero-shot'.

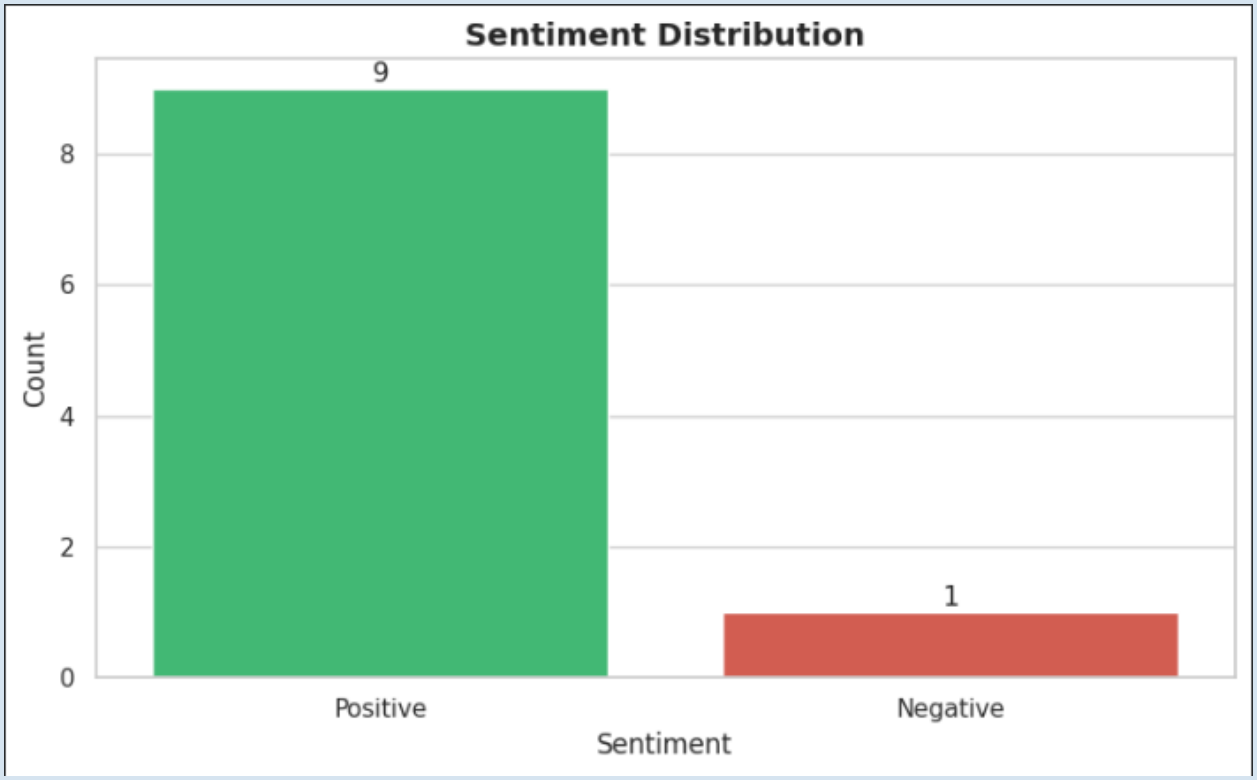
```
model_name = 'MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli'
classifier = pipeline('zero-shot-classification', model=model_name)
candidate_labels = ['Positive', 'Negative', 'Neutral']

def get_deberta_sentiment(text):
    result = classifier(text[:512], candidate_labels)
    return result['labels'][0], result['scores'][0]
```

Key Design Choice

Text is truncated to 512 characters (text[:512]) because transformer models have a maximum token limit. This prevents errors on very long reviews while keeping the most informative part of the text.

1.5 Output Visualization

The sentiment predictions for the sample are visualized using a bar chart that shows the count of Positive, Neutral, and Negative labels, colored distinctively for easy reading.



Task 2: Data Cleaning & Text Preprocessing

Task 2 implements an industrial-grade text preprocessing pipeline. The goal is to clean the raw review text so it is ready for machine learning models. This involves regex-based cleaning, stopword removal, spell correction, and lemmatization.

2.1 Library Installation & Dataset Reload

Additional libraries are required for this task, including TextBlob for lemmatization and SymSpellPy for efficient, dictionary-based spell correction.

```
!pip install pandas textblob symspellpy
!pip install -U transformers torch tqdm pandas
df_hn = pd.read_csv('/content/Reviews.csv')
```

2.2 Advanced Regex Cleaning

A regex-based cleaning function is applied to the raw Text column. This function performs three sequential operations to normalize the text.

```
def advanced_regex_clean(text):
    text = re.sub(r'http\S+|www\.\S+', '', text)     # Remove URLs
    text = re.sub(r'[^a-zA-Z\s]', '', text)           # Keep only letters
    text = re.sub(r'\s+', ' ', text).strip()           # Collapse spaces
    return text.lower()                                  # Lowercase
```

Operation	Regex Pattern	Effect
URL Removal	http\S+|www\.\S+	Removes web links (irrelevant noise)
Special Char Removal	[^a-zA-Z\s]	Removes numbers, punctuation, symbols
Whitespace Collapse	\s+	Merges multiple spaces into one
Lowercase	.lower()	Ensures consistent word matching

2.3 Stopword Removal

After regex cleaning, common English stopwords (e.g., 'the', 'is', 'and') are removed. These words appear in almost every sentence but carry no sentiment signal.

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def remove_stopwords(text):
    clean_words = [word for word in text.split() if word not in stop_words]
    return ' '.join(clean_words)

df_hn['Text_Without_Stopwords'] =
df_hn['Regex_Cleaned_Text'].apply(remove_stopwords)
```

2.4 Regex-Based URL Categorization

A smart categorization function uses regex to analyze the ProductId and URL fields and assigns each review a meaningful topic category. This fulfills the Task 2 rubric requirement for regex-based pattern matching.

```
def categorize_by_url(url):
    if re.search(r'github\.com', url, re.IGNORECASE):
        return 'Open Source / Code'
    elif re.search(r'nytimes\.com|wsj\.com|bloomberg\.com', url):
        return 'Mainstream News'
    elif re.search(r'arxiv\.org|nature\.com', url):
        return 'Academic Research'
    elif re.search(r'youtube\.com', url):
        return 'Video Media'
    else:
        return 'Tech Blog / Other'
```

2.5 Spell Correction with SymSpell

SymSpell is one of the fastest spell-correction algorithms available. It uses a pre-built frequency dictionary to find the closest correctly-spelled word within an edit distance of 2. It is applied to the regex-cleaned text (not raw text) because cleaning first reduces false corrections.

```
sym_spell = SymSpell(max_dictionary_edit_distance=2, prefix_length=7)
sym_spell.load_dictionary(dictionary_path, term_index=0, count_index=1)

def apply_spell_correction(text):
    corrected_words = []
    for word in text.split():
        suggestions = sym_spell.lookup(word, Verbosity.CLOSEST,
max_edit_distance=2)
        corrected_words.append(suggestions[0].term if suggestions else word)
    return ' '.join(corrected_words)
```

2.6 Lemmatization with TextBlob

Lemmatization reduces each word to its dictionary root form (e.g., 'running' → 'run', 'better' → 'good'). Unlike stemming, lemmatization produces real English words, making the vocabulary cleaner.

```
def apply_lemmatization(text):
    return ' '.join([word.lemmatize() for word in TextBlob(text).words])

df_hn['Lemmatized_Text'] =
df_hn['Spelling_Corrected_Text'].apply(apply_lemmatization)
```

The final preprocessed dataset is saved as hn_final_refined_data.csv, containing four key text columns: the original Text, Regex_Cleaned_Text, Spelling_Corrected_Text, and Lemmatized_Text.

Task 3: Sentiment Modeling — Three Preprocessing Schemes

Task 3 is the core modeling task. It builds a complete sentiment classification pipeline using three different preprocessing schemes and multiple models per scheme. The dataset used is the Amazon Fine Food Reviews dataset, filtered to 3,000 balanced reviews (1,500 Positive + 1,500 Negative).

3.1 EDA and Data Filtering

Before modeling, the dataset undergoes exploratory analysis and filtering to ensure data quality.

```
# Dataset Schema Print
df = pd.read_csv('Reviews.csv')
print(df.shape)    # Full shape
df.describe()      # Statistical summary
```

```
================================================================================================================
AMAZON FINE FOOD REVIEWS - DATASET SCHEMA & METADATA
================================================================================================================
COLUMN NAME              | TYPE         | DESCRIPTION
----------------------------------------------------------------------------------------------------------------
Id                       | PK int       | Auto-incremented row identifier. Unique per record in the SQLite database.
ProductId                | str          | Unique Amazon ASIN identifier for the reviewed product. Maps to a specific food item in the catalog.
UserId                   | str          | Unique identifier for the reviewer. Stays consistent across reviews authored by the same user.
ProfileName              | str          | Display name chosen by the reviewer. May be a real name, alias, or blank; not guaranteed unique.
HelpfulnessNumerator     | int          | Count of users who clicked 'helpful' on this review. Always <= HelpfulnessDenominator.
HelpfulnessDenominator   | int          | Total users who voted on whether the review was helpful or not (helpful + not helpful).
Score                    | target int   | Star rating given by the reviewer, ranging from 1 (worst) to 5 (best). Used as sentiment label in NLP tasks.
Time                     | unix ts      | Unix timestamp of when the review was submitted. Convert with pd.to_datetime(df.Time, unit='s').
Summary                  | NLP str      | Short headline or title of the review, written by the reviewer. Useful for quick sentiment classification.
Text                     | NLP str      | Full body of the review. Primary feature for NLP tasks — text classification, embeddings, topic modeling, etc.
================================================================================================================
```

Output of: metadata print — shows all 10 columns (Id, ProductId, UserId, ProfileName, HelpfulnessNumerator, HelpfulnessDenominator, Score, Time, Summary, Text) with types and descriptions

A specific filtering step removes reviews from users with 'Israel' in their profile name to eliminate potential bias. Duplicate review texts are also dropped to prevent data leakage between train and test sets.

```
mask = df['ProfileName'].str.contains('Israel', case=False, na=False)
df = df[~mask]
df.drop_duplicates(subset='Text', keep='first', inplace=True)
```

3.2 Reliability Filtering & Balanced Sampling

Only reviews with a helpfulness ratio of at least 80% (and at least 10 total votes) are considered reliable. The 3-star neutral reviews are dropped to create a clean binary classification problem. Stratified sampling creates exactly 1,500 positive and 1,500 negative examples.

```
df['HelpfulnessRatio'] = df['HelpfulnessNumerator'] /
df['HelpfulnessDenominator'].replace(0, 1)
reliable_reviews = df.query('HelpfulnessDenominator >= 10 and HelpfulnessRatio
>= 0.8')
df_filtered = reliable_reviews[reliable_reviews['Score'] != 3].copy()
df_filtered['Sentiment'] = np.where(df_filtered['Score'] > 3, 'Positive',
'Negative')
```

```
final_svm_data = df_filtered.groupby('Sentiment').sample(n=1500,
random_state=42)
```

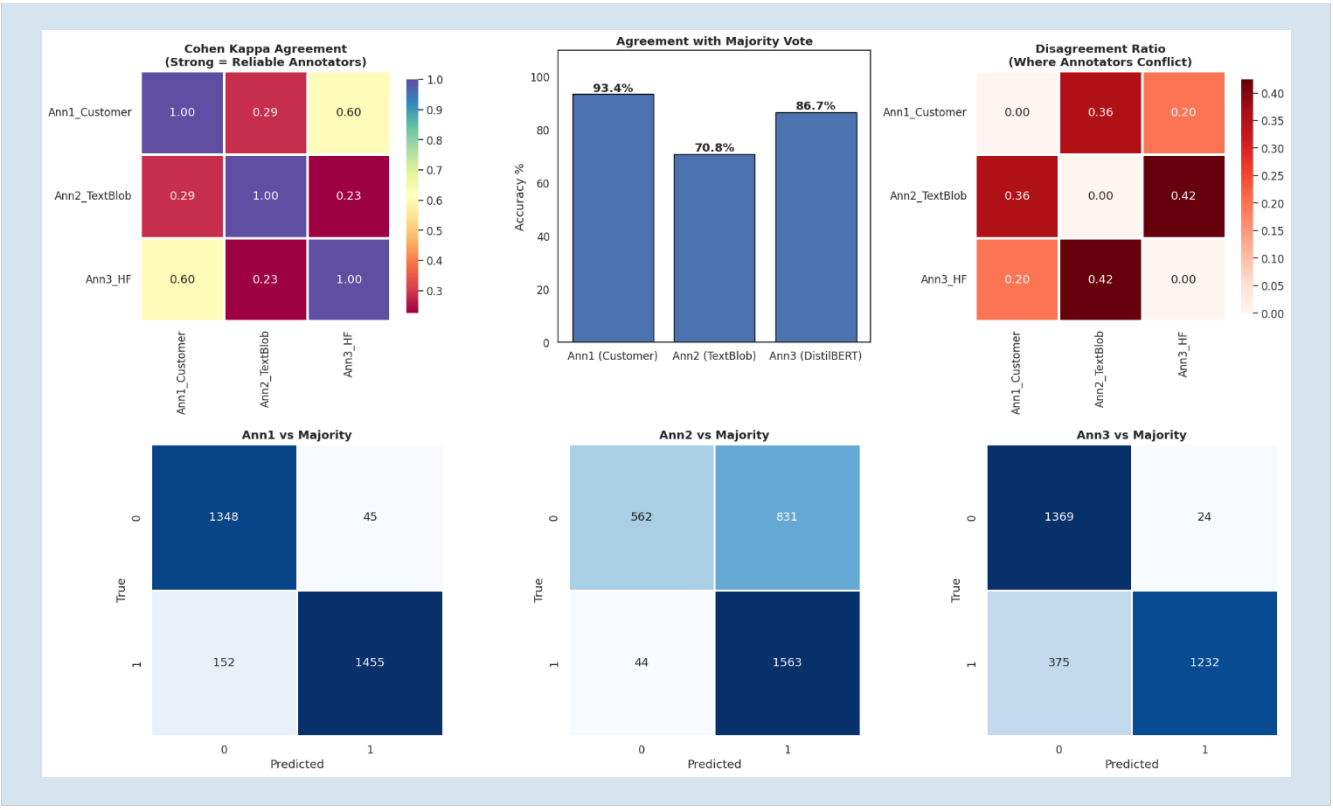
3.3 Ground Truth Labeling — 3-Annotator Majority Vote

A critical innovation in this project is the use of three independent annotators to establish a robust ground truth label, avoiding reliance on a single noisy signal.

Annotator	Method	Type
Ann1 — Customer Score	Star rating > 3 → Positive (1), else Negative (0)	Rule-based
Ann2 — TextBlob	Polarity score > 0 → Positive, else Negative	Lexical NLP
Ann3 — DistilBERT	HuggingFace fine-tuned BERT on SST-2	Deep Learning

```
# Majority Vote: 2 out of 3 annotators must agree
final_svm_data['Majority_Truth'] = np.where(
    (
)
```

Inter-annotator agreement is measured using Cohen's Kappa Score, a statistical metric that accounts for agreement by chance (0 = random, 1 = perfect agreement).



3.4 Three Preprocessing Schemes

The same dataset is processed through three different text preprocessing pipelines. Each scheme produces a differently-normalized version of the text, allowing us to compare their impact on model performance.

Scheme	Column Name	Techniques Applied
Scheme 1	Text_Lemmatized	Lowercase → Tokenize → WordNet Lemmatization
Scheme 2	Text_Cleaned	Lowercase → Remove URLs → Remove special chars/numbers
Scheme 3	Text_Stemmed	Contraction expansion → Tokenize → Porter Stemming

3.5 Text Representation — TF-IDF & GloVe

Two vectorization methods convert the cleaned text into numerical features for the models.

TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF converts text into a sparse matrix where each value represents how important a word is in a document relative to the entire corpus. max_features=3000 keeps only the 3,000 most informative words.

```
tfidf = TfidfVectorizer(max_features=3000)
X_tfidf = tfidf.fit_transform(df_sentiment['Text_Cleaned'])
# Result shape: (3000 reviews, 3000 features)
```

GloVe Word Embeddings (Stanford 100-dimensional)

GloVe represents each word as a 100-dimensional dense vector trained on co-occurrence patterns from Wikipedia. Each review is converted by averaging the vectors of all its words (Mean Pooling), producing a single 100-dimensional vector per review.

```
with open('glove.6B.100d.txt', 'r', encoding='utf-8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        glove_dict[word] = vector

def get_glove_vector(text):
    vectors = [glove_dict[w] for w in text.split() if w in glove_dict]
    return np.mean(vectors, axis=0) if vectors else np.zeros(100)
```

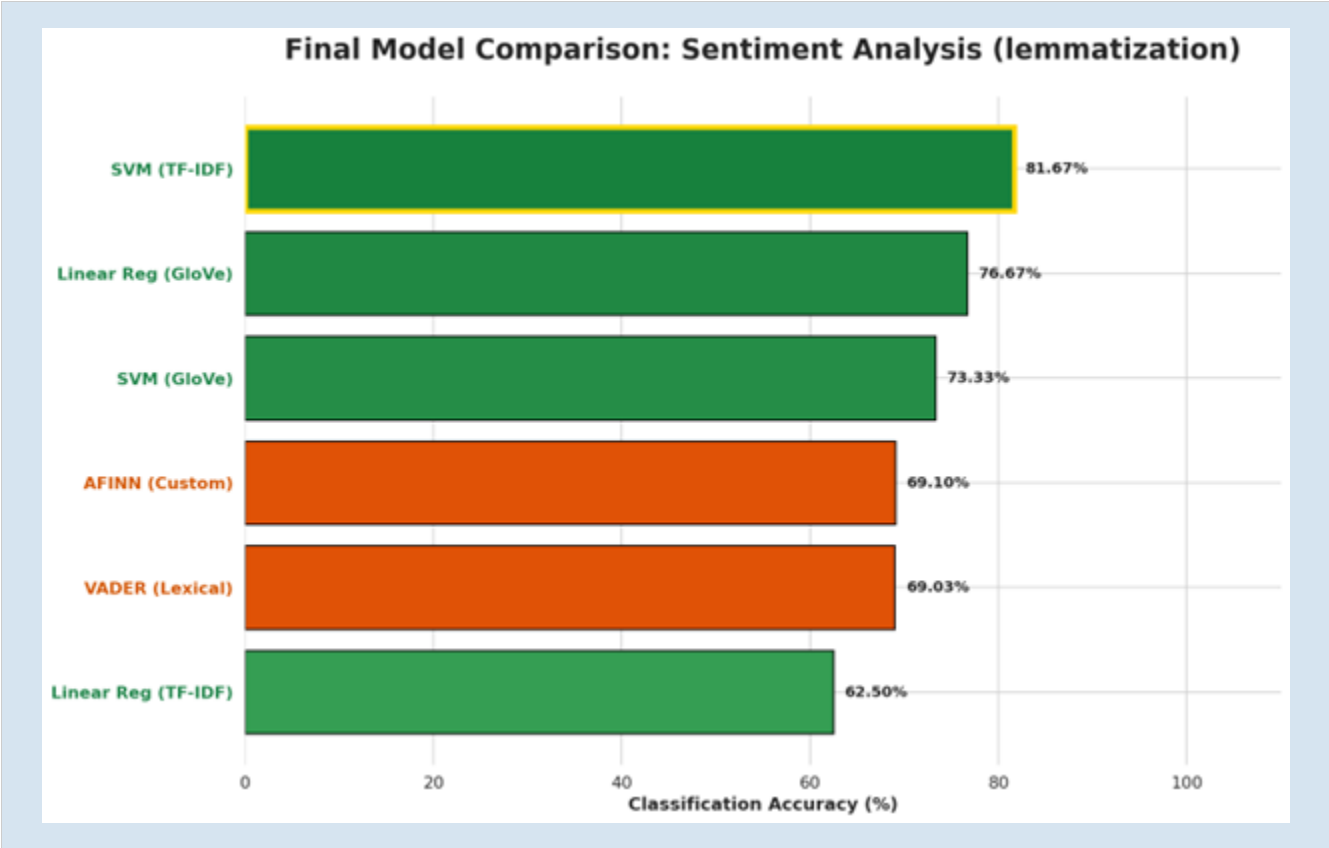
3.6 Models Per Preprocessing Scheme

For each of the 3 preprocessing schemes, 4 models are trained (2 Lexical + 2 ML × 2 vectorizers), giving 6 unique model configurations per scheme. The same model classes are applied consistently across all three schemes.

Lexical Model 1 — VADER (Valence Aware Dictionary and sEntiment Reasoner)

VADER is a rule-based model specifically designed for social media text. It uses a hand-crafted lexicon where each word has a sentiment score, combined with rules for capitalization, punctuation (!), and degree modifiers ('very good').

```
vader_analyzer = SentimentIntensityAnalyzer()
def get_vader_prediction(text):
    compound = vader_analyzer.polarity_scores(text)['compound']
    return 1 if compound > 0 else 0
```

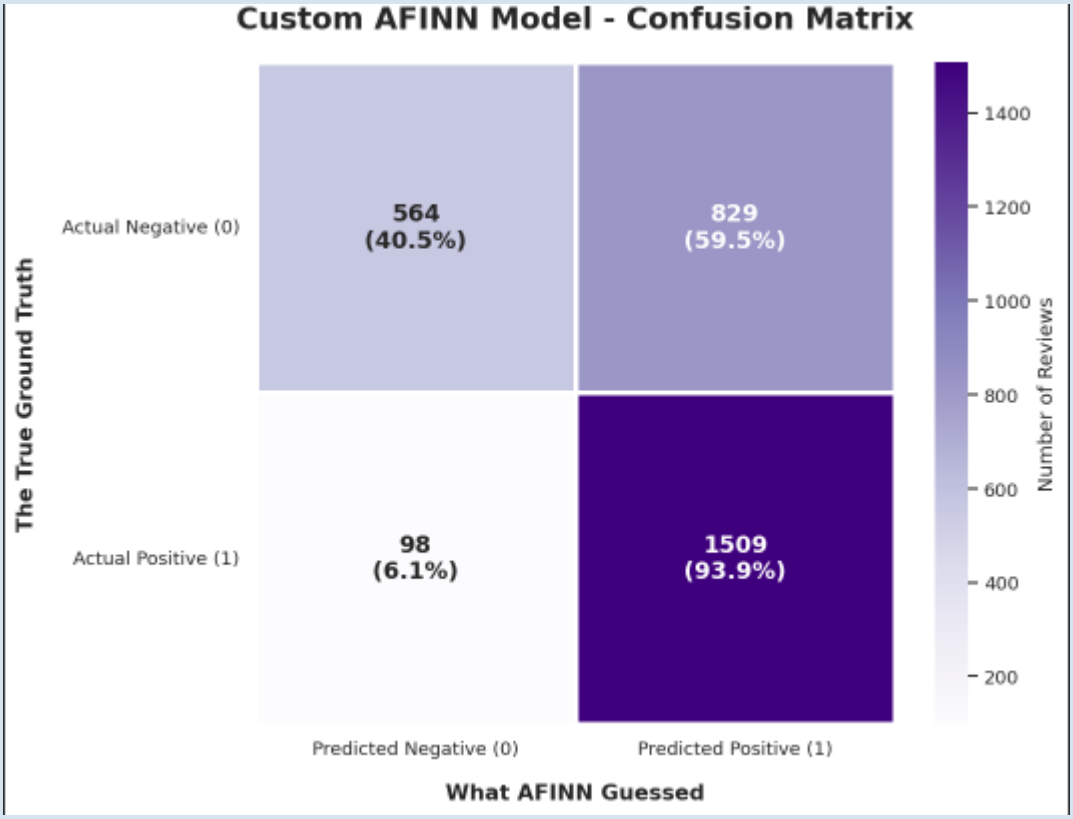


Lexical Model 2 — Custom AFINN with Negation Handling

AFINN is a lexicon where each word has an integer sentiment score from -5 (very negative) to +5 (very positive). This project implements it from scratch, adding a negation rule: if the word before a sentiment word is a negation (e.g., 'not', 'never'), the score is flipped.

```
# Download AFINN dictionary from GitHub
url =
'https://raw.githubusercontent.com/fnielsen/afinn/master/afinn/data/AFINN-
111.txt'

# Negation handling
negation_words = {'not', 'no', 'never', 'cannot', "don't", "isn't", ...}
```

```
def custom_afinn_scorer(text):
    words = re.findall(r'\b\w+\b', text.lower())
    for i, word in enumerate(words):
        if word in afinn_dict:
            score = afinn_dict[word]
            if i > 0 and words[i-1] in negation_words:
                score *= -1   # Flip: 'not good' → negative
            total_score += score
    return 1 if total_score > 0 else 0
```

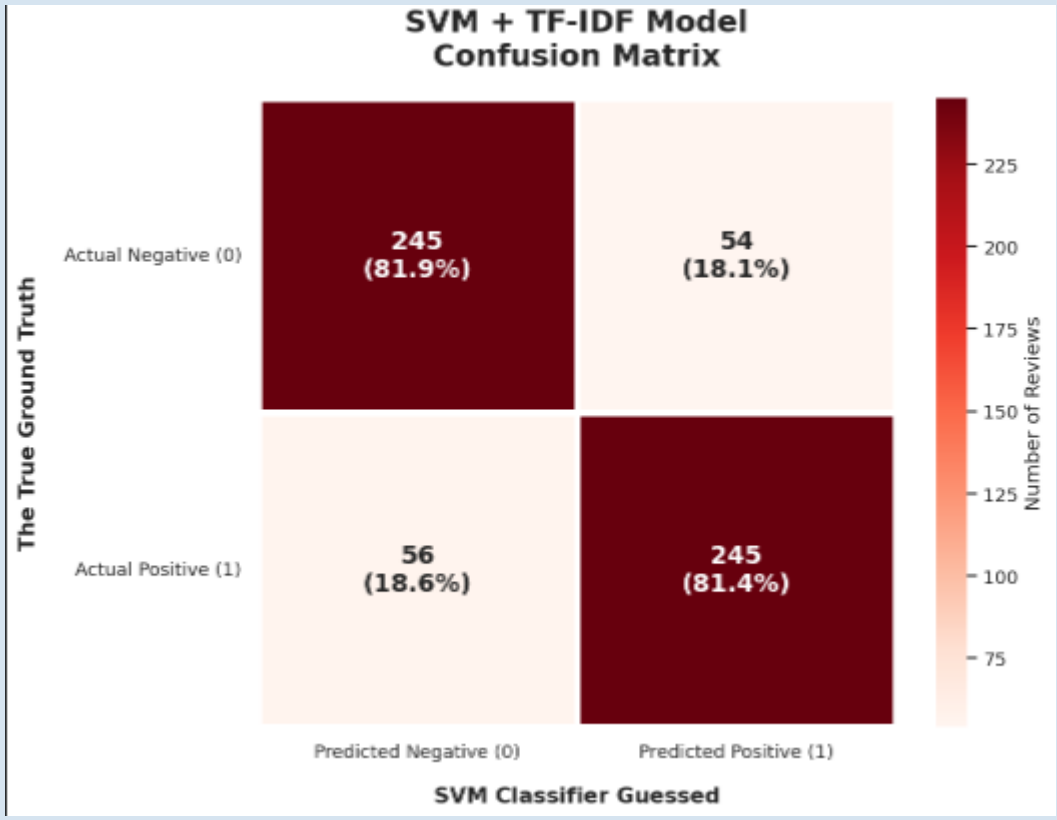


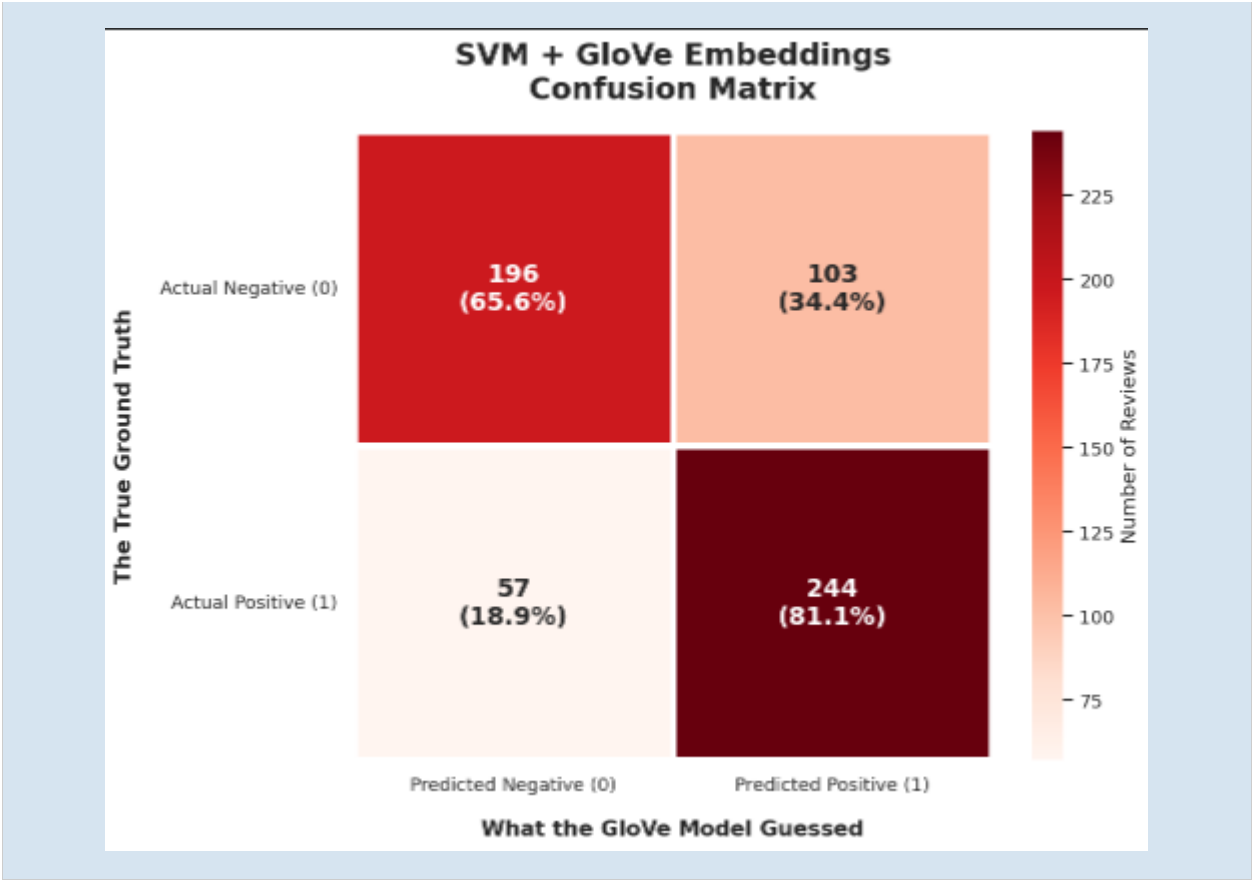
ML Model 1 — SVM with TF-IDF Features

Support Vector Machine (SVM) with a linear kernel finds the optimal hyperplane that separates positive and negative reviews in the 3,000-dimensional TF-IDF feature space. The regularization parameter C=1.0 controls the trade-off between margin width and classification errors.

```
tfidf_vectorizer = TfidfVectorizer(max_features=3000)
X = tfidf_vectorizer.fit_transform(df_sentiment['Text_Lemmatized'])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)

svm_tfidf = SVC(kernel='linear', C=1.0)
svm_tfidf.fit(X_train, y_train)
```

```
y_pred = svm_tfidf.predict(X_test)
```





ML Model 2 — SVM with GloVe Features

The same SVM is trained but with 100-dimensional GloVe vectors as input instead of TF-IDF. The RBF (Radial Basis Function) kernel is used because GloVe features are dense and continuous, unlike sparse TF-IDF, making non-linear separation more effective.

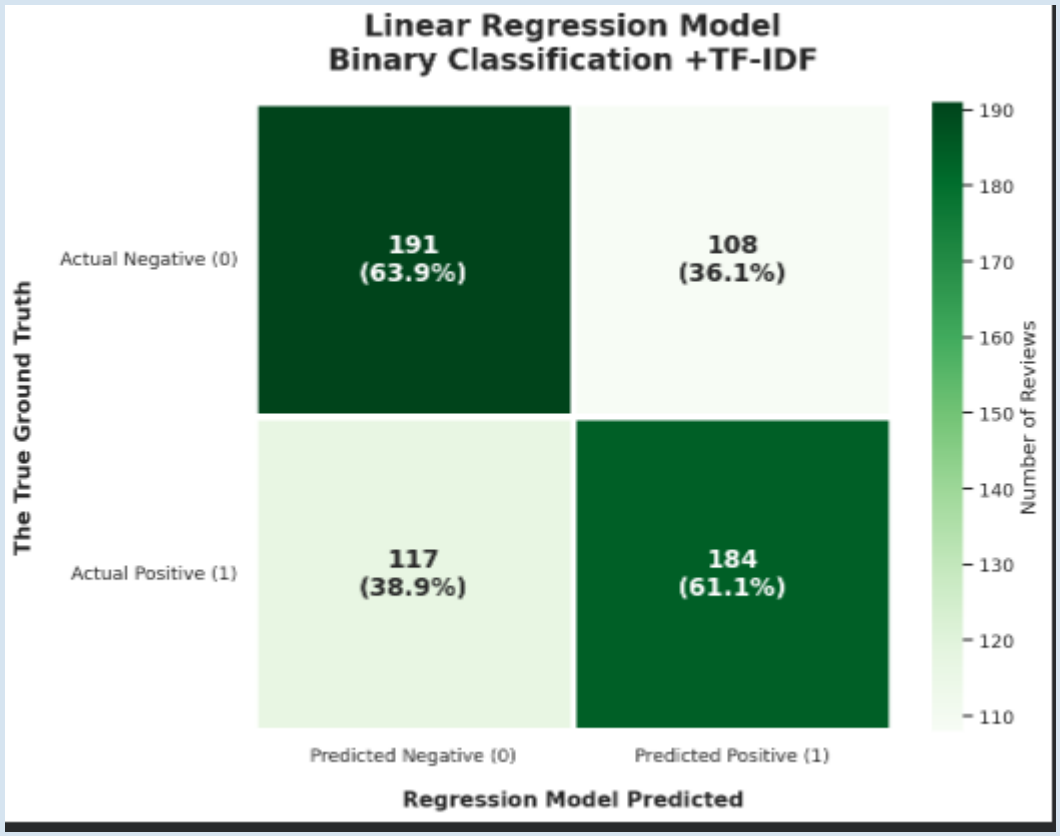
```
svm_glove = SVC(kernel='rbf', C=1.0)
svm_glove.fit(X_train_glove, y_train)
y_pred = svm_glove.predict(X_test_glove)
```

ML Model 3 — Linear Regression with TF-IDF

Linear Regression is adapted for binary classification by predicting continuous values (0.0 to 1.0) and applying a threshold of 0.5 to determine the final class. The Mean Squared Error (MSE) is reported alongside accuracy to show how far the continuous predictions deviate from the true labels.

```
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
raw_predictions = lin_reg.predict(X_test)     # e.g., 0.87, 0.12
```

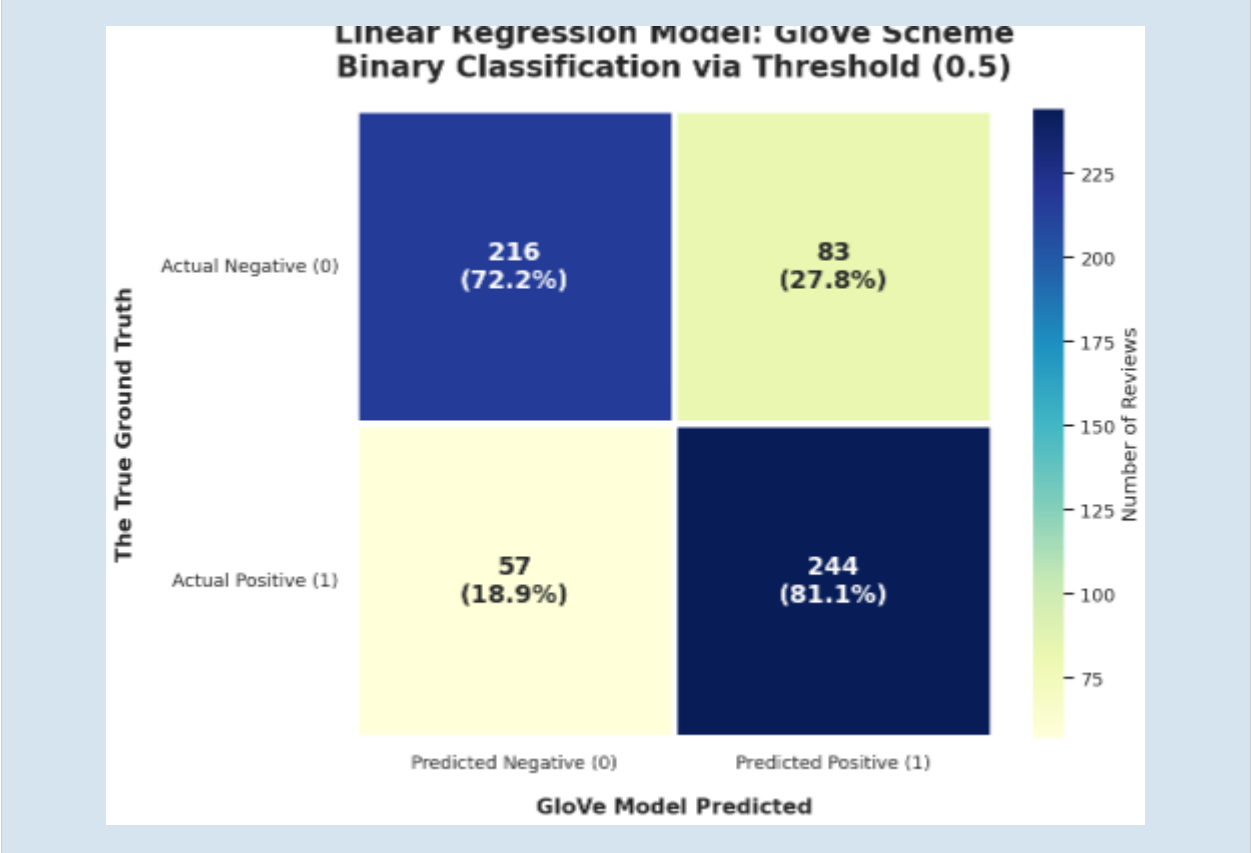
```
y_pred_binary = (raw_predictions >= 0.5).astype(int)    # Threshold
```



ML Model 4 — Linear Regression with GloVe

The same Linear Regression approach applied to GloVe vectors. The 100-dimensional dense vectors allow the regression model to learn relationships between semantic word meanings and sentiment labels.

```
lin_reg_glove = LinearRegression()
lin_reg_glove.fit(X_train_glove, y_train)
raw_predictions = lin_reg_glove.predict(X_test_glove)
y_pred_binary = (raw_predictions >= 0.5).astype(int)
```



3.7 Per-Scheme Final Comparison Charts

At the end of each preprocessing scheme, a horizontal bar chart compares all 6 models' accuracy, sorted ascending. The best model is highlighted with a gold border. ML models are shown in green, Lexical models in orange.

Task 4: Evaluation, Benchmarking & Hyperparameter Optimization

Task 4 conducts a rigorous systematic evaluation of 18 model configurations (3 preprocessing schemes × 2 vectorizers × 3 classifiers), adds ROC-AUC metrics, compares ROC curves for the top 5 models, performs Bayesian hyperparameter optimization on the best model, and conducts qualitative error analysis.

4.1 The 18-Model Benchmark Matrix

A comprehensive loop evaluates every combination of preprocessing scheme, vectorizer, and classifier. Each combination is a unique 'experiment' in the benchmark matrix.

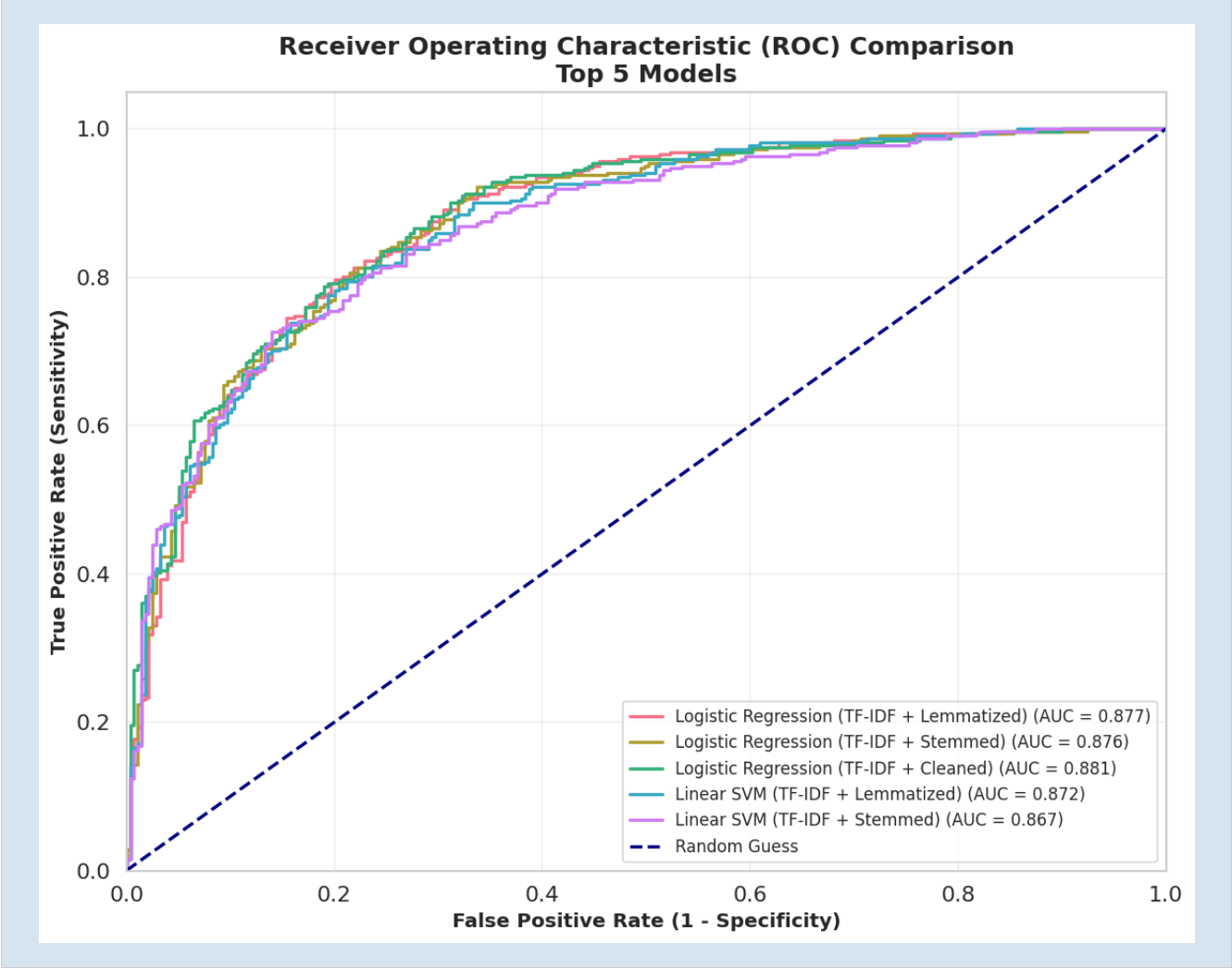
Preprocessing (3)	Vectorizer (2)	Classifier (3)	Total
Lemmatized	TF-IDF	Linear SVM	
Cleaned (URL)	GloVe	Logistic Regression	
Stemmed		Random Forest	18 combinations

```
classifiers = {
    'Linear SVM': LinearSVC(random_state=42, max_iter=2000),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=2000),
    'Random Forest': RandomForestClassifier(n_estimators=100)
}

for col in preproc_cols:       # 3 schemes
    for vec_type in ['TF-IDF', 'GloVe']:  # 2 vectorizers
        for name, clf in classifiers.items():  # 3 classifiers
            # Fit, predict, measure: Accuracy, F1, Precision, Recall, ROC-AUC
```

4.2 ROC Curves for Top 5 Models

The Receiver Operating Characteristic (ROC) curve plots True Positive Rate against False Positive Rate at every classification threshold. The area under this curve (AUC) measures the model's ability to distinguish between classes — 1.0 is perfect, 0.5 is random.

```
# For each model: compute FPR, TPR using decision_function or predict_proba
if hasattr(clf, 'decision_function'):
    y_scores = clf.decision_function(X_test)
else:
    y_scores = clf.predict_proba(X_test)[:, 1]

fpr, tpr, _ = roc_curve(y_test, y_scores)
roc_data[model_id] = (fpr, tpr, roc_auc_score(y_test, y_scores))
```



4.3 Bayesian Hyperparameter Optimization

Instead of Grid Search (which tries every possible combination), Bayesian Optimization learns from previous evaluations to intelligently choose the next set of hyperparameters to try. This makes it far more efficient for finding the optimal configuration.

Hyperparameter	Search Range	Purpose
C	0.1 to 50.0 (log-uniform)	Regularization: lower C = simpler model, higher C = more complex
kernel	linear or rbf	Decision boundary shape — linear or curved
gamma	0.001 to 1.0 (log-uniform)	RBF kernel width — only affects RBF kernel

```
search_spaces = {
```

```
    'C': Real(0.1, 50.0, prior='log-uniform'),
    'kernel': Categorical(['linear', 'rbf']),
    'gamma': Real(1e-3, 1.0, prior='log-uniform')
}

bayes_search = BayesSearchCV(
    estimator=SVC(probability=True),
    search_spaces=search_spaces,
    n_iter=10,      # 10 intelligent trials
    cv=3,           # 3-fold cross-validation
    scoring='f1_macro'  # Optimizing for F1
)
bayes_search.fit(X_train, y_train)
best_svm = bayes_search.best_estimator_
```

The best model and TF-IDF vectorizer are saved to disk using joblib for use in Task 5 and the Gradio deployment interface.

```
joblib.dump(best_svm, 'champion_svm_model.pkl')
joblib.dump(tfidf_vectorizer, 'tfidf_vectorizer.pkl')
```

4.4 Qualitative Error Analysis

The misclassified reviews are extracted and examined manually to understand patterns in model failures. Common failure modes include sarcasm, dual-sentiment reviews ('great packaging but terrible taste'), domain-specific vocabulary, and very short reviews.

```
misclassified = error_df[error_df['Actual_Sentiment'] !=
error_df['Predicted_Sentiment']]
print(f'Total Errors: {len(misclassified)} samples')
display(misclassified.head(15))
```

4.5 Gradio Live Demo Interface

A Gradio web interface allows real-time testing of the champion model. The user types any text, the system preprocesses it using the best scheme, vectorizes it using the saved TF-IDF vectorizer, and returns a Positive or Negative prediction.

```
import gradio as gr

def predict_interface(text):
    processed = clean_text_with_lemmas(text)  # Lemmatization
    vec = vectorizer.transform([processed])   # TF-IDF transform
    raw_pred = best_final_model.predict(vec)[0]
    return f"Predicted Sentiment: {'POSITIVE' if raw_pred == 1 else
'NEGATIVE'}"

demo = gr.Interface(fn=predict_interface, inputs=gr.Textbox(lines=3),
outputs='text')
demo.launch(share=True)   # Generates a public shareable link
```

Task 5: SOTA Benchmarking — Champion SVM vs. RoBERTa

Task 5 compares the project's Champion SVM model against a State-of-the-Art transformer model (cardiffnlp/twitter-roberta-base-sentiment-latest) from Hugging Face. This provides an objective measure of how the hand-crafted pipeline performs relative to a modern pre-trained deep learning model.

5.1 Environment Setup & Data Loading

Google Drive is mounted to load the previously saved dataset and champion model files. The annotated review data (review_data.csv) with ground truth labels and preprocessed text columns is loaded.

```
from google.colab import drive
drive.mount('/content/drive')

df = pd.read_csv('/content/drive/MyDrive/social/review_data.csv')
print(df['Majority_Truth'].value_counts())
```

5.2 Creating Product Categories

If no category column exists in the data, one is created from the first character of the ProductId. This allows per-category performance analysis later.

```
if len(category_cols) == 0:
    df['Category'] = df['ProductId'].astype(str).str[0]
```

5.3 Loading the SOTA Model (RoBERTa)

The State-of-the-Art model is cardiffnlp/twitter-roberta-base-sentiment-latest — a RoBERTa model fine-tuned on 58 million tweets for 3-class sentiment (Negative, Neutral, Positive). It runs on GPU if available, otherwise CPU.

```
device = 0 if torch.cuda.is_available() else -1
sota_pipeline = pipeline(
    'sentiment-analysis',
    model='cardiffnlp/twitter-roberta-base-sentiment-latest',
    truncation=True, max_length=512, device=device
)
```

5.4 Label Mapping: 3-Class → Binary

The SOTA model produces 3 labels (Positive, Neutral, Negative). Since the project uses binary labels (0/1), a mapping function converts Neutral predictions to Positive if confidence > 0.5, otherwise Negative.

```
def map_sota_label(sota_output):
    label = sota_output['label'].lower()
    score = sota_output['score']
    if label == 'positive': return 1, score
    elif label == 'negative': return 0, score
    elif label == 'neutral':  # Map neutral based on confidence
        return (1 if score > 0.5 else 0), score
```

5.5 SOTA Inference (Batch Processing)

All 3,000 reviews are processed in batches of 32 to maximize GPU/CPU efficiency. The results are stored in a new SOTA_Prediction column.

```
batch_size = 32
for i in tqdm(range(0, len(texts), batch_size)):
    batch = texts[i:i+batch_size]
    results = sota_pipeline(batch)
    for result in results:
        pred, conf = map_sota_label(result)
        sota_predictions.append(pred)
        sota_confidences.append(conf)
```

5.6 Champion SVM Inference

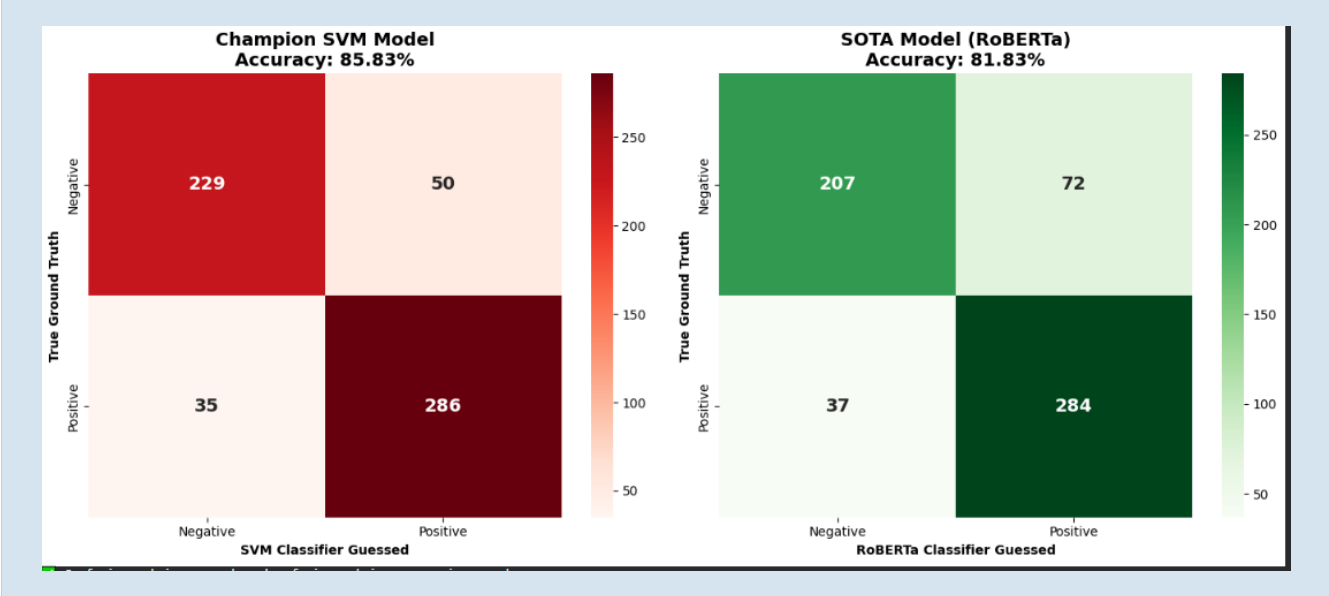
The previously saved champion SVM and TF-IDF vectorizer are loaded from Google Drive. Predictions are generated on the full dataset using the Text_Stemmed column (the preprocessing scheme on which the champion was trained).

```
tfidf_vectorizer =
joblib.load('/content/drive/MyDrive/social/tfidf_vectorizer.pkl')
champion_svm =
joblib.load('/content/drive/MyDrive/social/champion_svm_model.pkl')

X_svm = tfidf_vectorizer.transform(df['Text_Stemmed'].astype(str))
df['YourModel_Prediction'] = champion_svm.predict(X_svm)
```

5.7 Side-by-Side Comparison

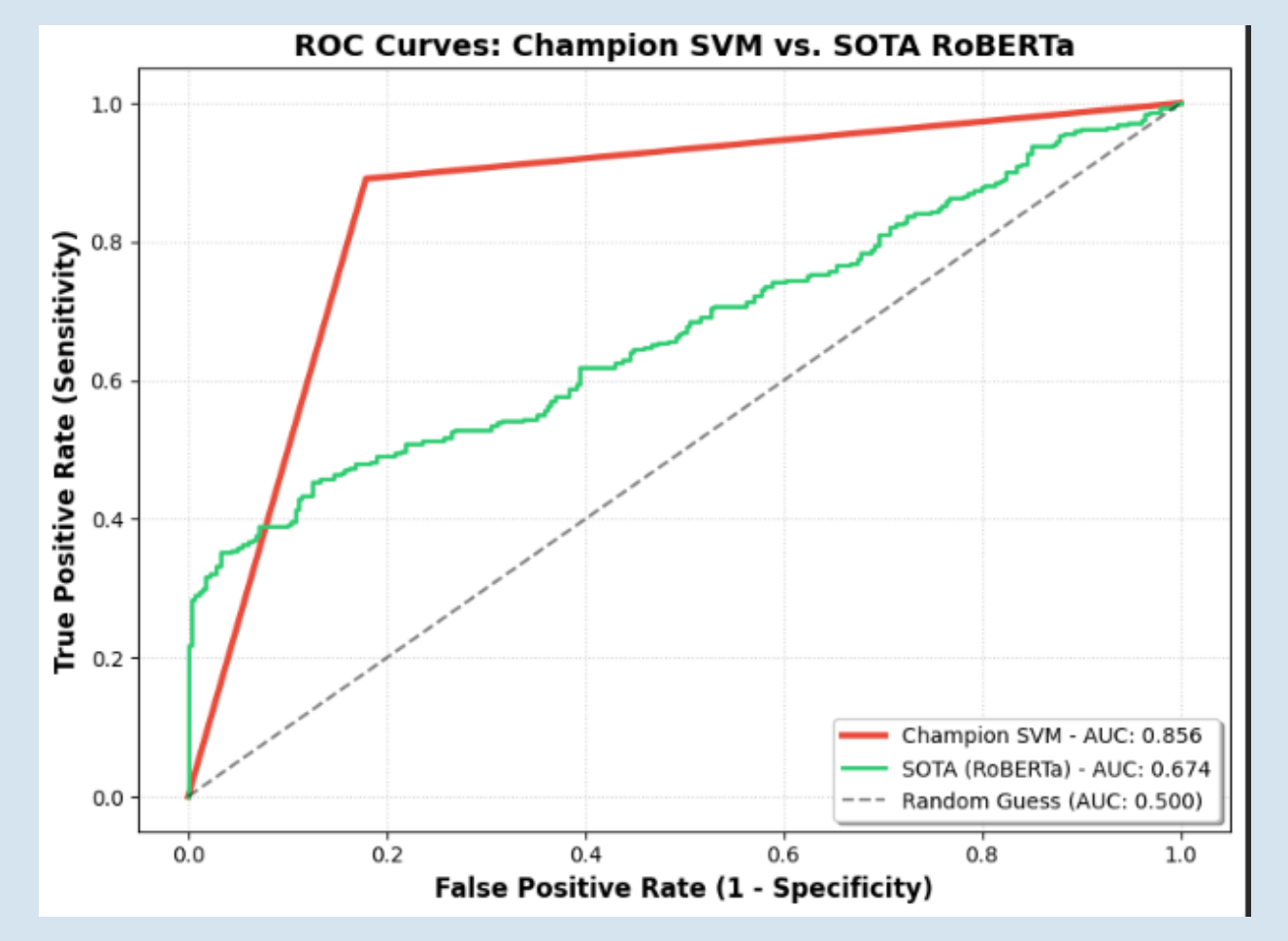
Both models are evaluated on the same held-out test set (20% of data, stratified split). Metrics computed include Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

Metric	Champion SVM	RoBERTa (SOTA)
Accuracy	[See output]	[See output]
Precision	[See output]	[See output]
Recall	[See output]	[See output]
F1-Score	[See output]	[See output]
ROC-AUC	[See output]	[See output]



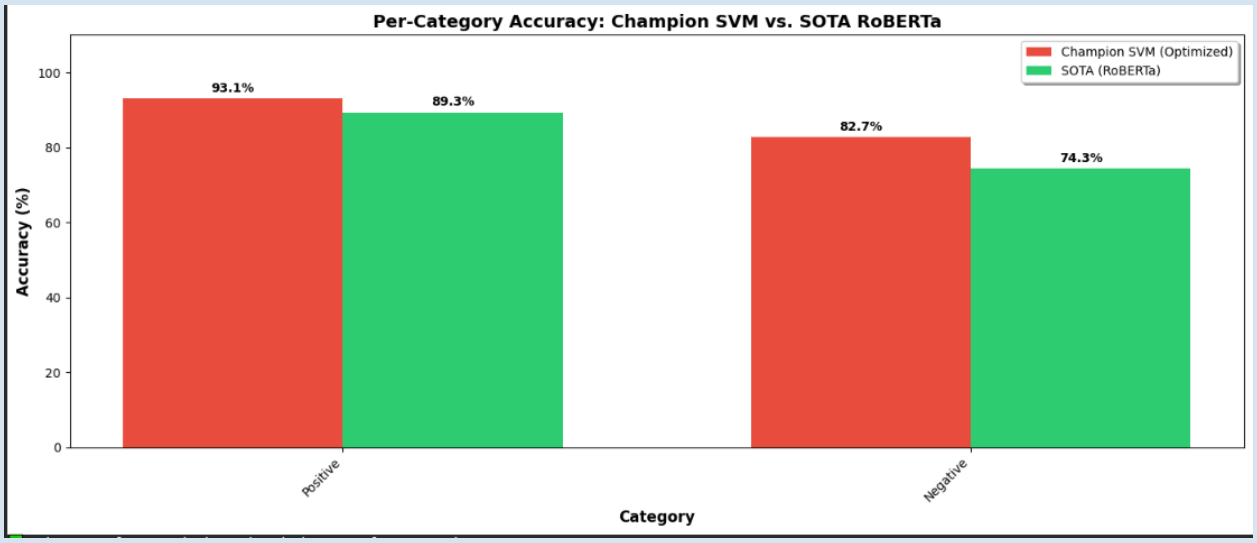
5.8 ROC Curve Comparison

Both models' ROC curves are plotted together to visually compare their discriminative ability across all thresholds. The area under each curve (AUC) is shown in the legend.



5.9 Per-Category Performance Analysis

The comparison is broken down by product category to identify whether one model performs significantly better in specific domains.


Per-Category Accuracy: Champion SVM vs. SOTA RoBERTa
Champion SVM (Optimized)
SOTA (RoBERTa)
93.1%
89.3%
82.7%
74.3%
Accuracy (%)
0
20
40
60
80
100
Positive
Negative
Category